

Design & Architecture

Project Lead: Aiden Cary (Project and Backend Lead)

Team Members: Keller Willhite (UI/UX Developer), Zachery Atchley (Integration and Unit Tester)

Date: March 3, 2026

1. Architectural Decisions

Repository Architecture

We have chosen a **Repository Architecture** to manage the complexity of code analysis. In this model, the **ASTObject** (Abstract Syntax Tree) acts as the central data store—or "Repository"—that all specialized engines (Static, Standards, and AI) read from. This ensures that our analysis components remain **decoupled**; we can add a new security engine without changing how the AI predictor works.

Web-Based Layered Architecture

CodePulse uses a **Layered Architecture** to separate concerns between the user interface and the heavy logic:

- **Presentation Layer (Frontend):** A JavaScript/React dashboard deployed on **Vercel** for real-time interaction.
 - **Service Layer (Backend):** A **FastAPI** (Python) server that orchestrates the analysis pipeline.
 - **Data Layer (Persistence):** A **Supabase/PostgreSQL** database that handles authentication and encrypted storage.
-

2. Design Models

Use Case Diagram

This model illustrates the functional requirements through interactions between our **Actors** and the system boundary.

- **Primary Actor (Developer):** Positioned on the left, the Developer initiates **UC-1 (Authenticate)**, **UC-2 (Submit Code)**, and **UC-3 (View Quality Report)**.
- **Secondary Actor (Supabase/PostgreSQL):** Positioned on the right, the database is involved in **UC-1 (Authenticate)** and **UC-4 (Retrieve History)** as the system must fetch stored data to fulfill these requests.

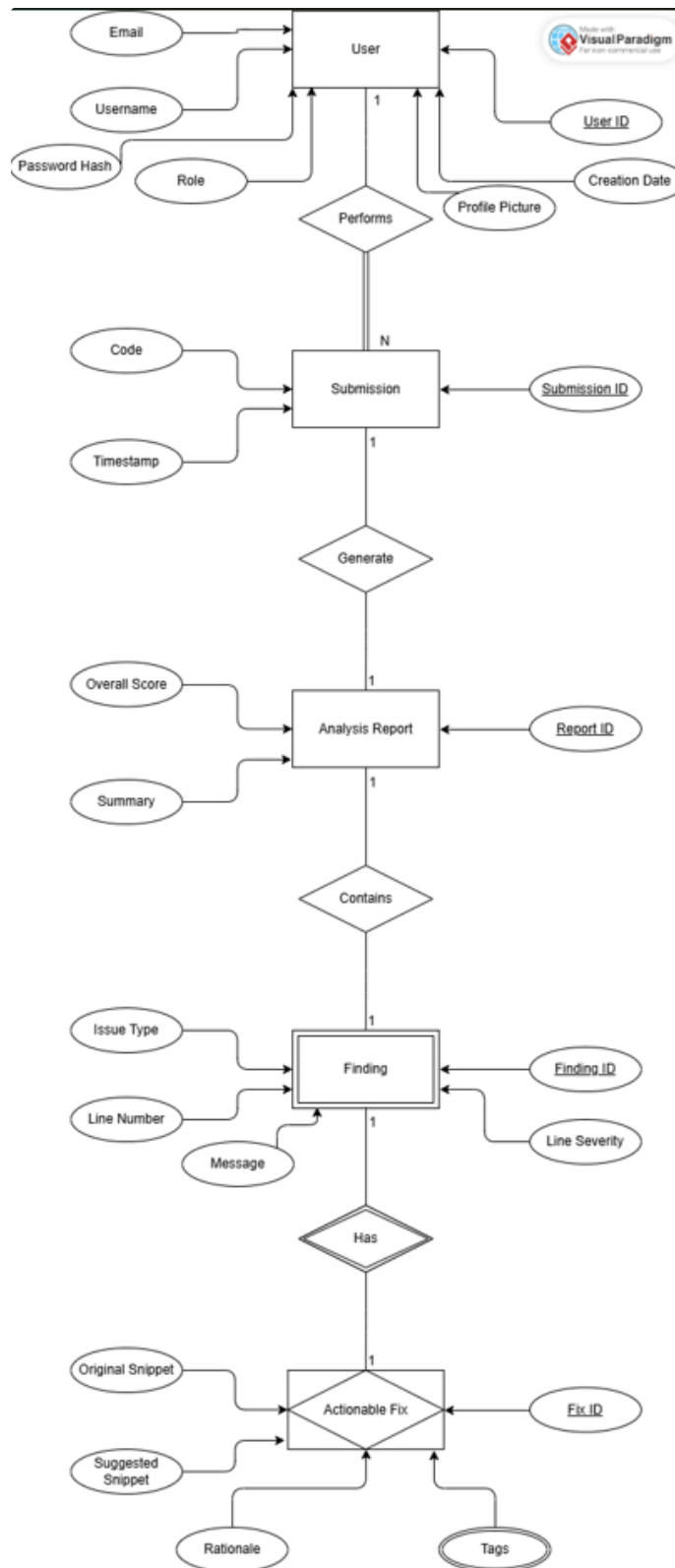
- **Specialized Actor (Team Lead):** A "favored user" who has exclusive access to a specific use case: **Generate Advanced Team Report**, designed to track technical debt across the organization.

ER Diagram (Data Model)

Our data model follows a **Transaction Processing** pattern, where each code submission triggers a series of related data entries.

- **User (Regular Entity):** Stores identity metadata like `email`, `username`, and `password_hash`.
- **Submission (Regular Entity):** Holds the `encrypted_code` and a `timestamp` of when it was sent.
- **Analysis Report (Regular Entity):** Contains the `overall_score` (a **derived attribute**) and a summary of the run.
- **Finding (Weak Entity):** Represents individual bugs or style issues. It is **weak** because it cannot exist without a parent `Analysis Report`.
- **Actionable Fix (Associative Entity):** Links a `Finding` to a specific solution, including the `suggested_snippet` and `rationale`.

Link to edit diagram: <https://online.visual-paradigm.com/w/awdyzwnx/diagrams/#diagram:workspace=awdyzwnx&proj=0&id=4&type=ChenEntityRelationshipDiagram>



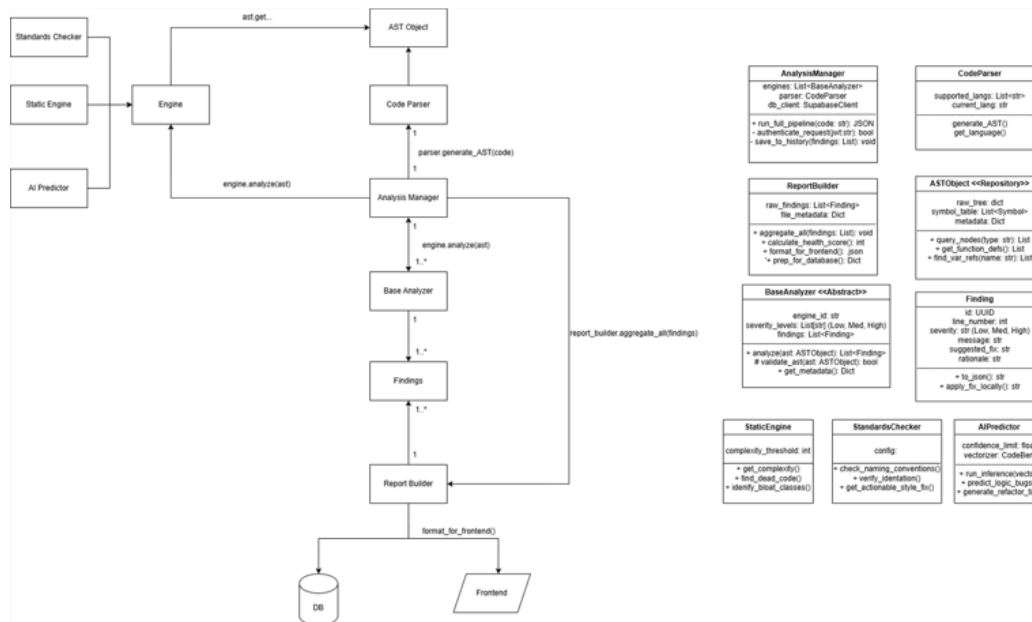
Class Diagram

The class diagram defines the structural "blueprint" of our Python backend.

- **AnalysisManager (Orchestrator):** The main entry point that calls the `CodeParser`, runs the `engines`, and triggers the `save_to_history()` private method.

- **BaseAnalyzer (Abstract Class):** Uses the `<<Abstract>>` stereotype to define a mandatory `analyze(ast)` contract for all child engines.
- **StaticEngine & AIPredictor:** Specialized components that perform deterministic checks (like complexity) and probabilistic checks (using **CodeBERT** pattern matching).
- **ReportBuilder:** Aggregates all identified **Findings** to calculate the final health score and format the data for the frontend.

Link to edit diagram: [https://app.diagrams.net/#G1tyd4IN-R7_g-Vmyv66j9_ZLiBOBH5dZX#{_pageId"%3A"rz56g_N6fbcdNOHA5R8j"}](https://app.diagrams.net/#G1tyd4IN-R7_g-Vmyv66j9_ZLiBOBH5dZX#{_pageId).

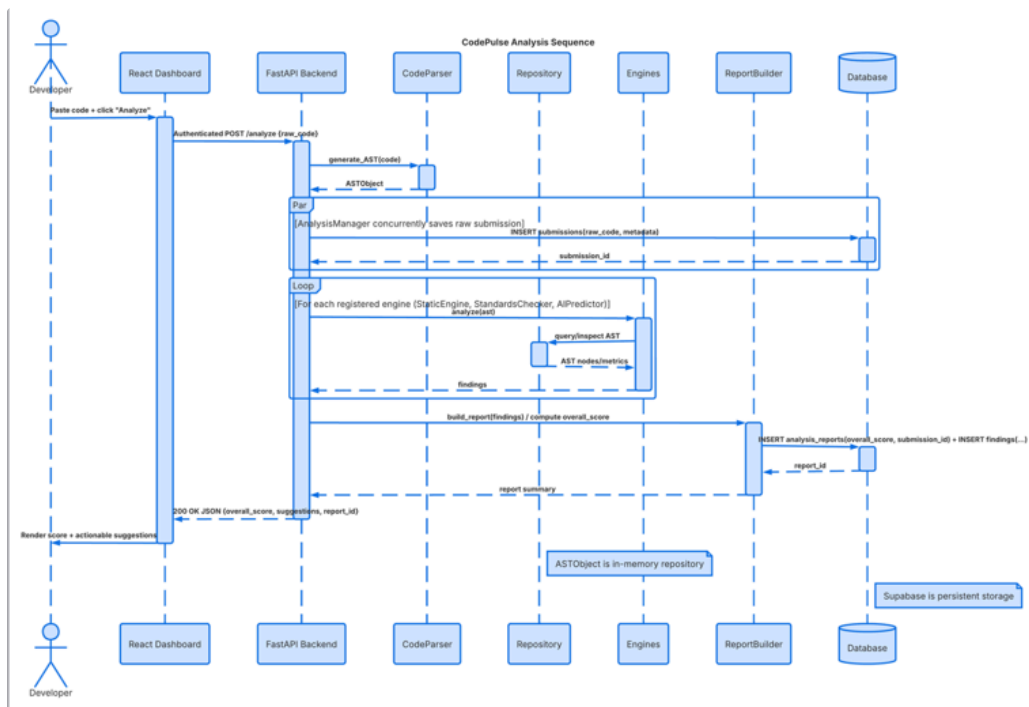


Sequence Diagram

1. **Trigger (Developer → Frontend):** The Developer pastes code into the React dashboard and clicks "Analyze".
2. **Request (Frontend → AnalysisManager):** The frontend sends an authenticated POST request containing the raw code string to the FastAPI backend.
3. **Parsing (AnalysisManager → CodeParser):** The Manager calls `generate_AST(code)`, which returns a structured **ASTObject** (the Repository).
4. **Persistence (AnalysisManager → Supabase):** The Manager concurrently saves the raw code to the `submissions` table to ensure data integrity.
5. **Analysis Loop (AnalysisManager → Engines):** The Manager iterates through the registered engines (`StaticEngine`, `StandardsChecker`, `AIPredictor`), calling

`analyze(ast)` on each.

6. **Data Retrieval (Engines → ASTObject):** Each engine "queries" the AST repository to identify structural patterns, complexity, or risky logic.
7. **Aggregation (AnalysisManager → ReportBuilder):** Once all engines finish, the findings are handed to the `ReportBuilder` to calculate the final `overall_score`.
8. **Storage (ReportBuilder → Supabase):** The finalized report and its individual findings are pushed to the `analysis_reports` and `findings` tables.
9. **Response (AnalysisManager → Frontend):** The backend returns a structured JSON payload.
10. **Render (Frontend → Developer):** The dashboard updates to display the quality score and actionable suggestions.



The Engine Pipeline Workflow

1. Orchestration Start

The process begins when the **AnalysisManager** (the orchestrator) initiates the execution loop. Rather than having each engine work in isolation, the manager ensures they are all fed the same data to maintain consistency across the report.

2. The ASTObject Repository (The Input)

Instead of reading the raw code string, every engine connects to the **ASTObject**, which serves as our **Repository**.

- **Shared Data:** The repository provides the parsed symbol table and structural metadata that was generated by the **CodeParser**.
- **Efficiency:** This prevents redundant processing, as we only parse the code once regardless of how many engines we run.

3. Parallel Analysis Engines

The flow then branches out into three specialized analysis paths that can run concurrently:

- **StaticEngine (Deterministic):** Focuses on structural integrity, such as calculating **Cyclomatic Complexity**, identifying **Dead Code**, and mapping the program's internal **Data Flow**.
- **StandardsChecker (Linter):** Specifically compares the code against the **PEP 8** standard, checking for naming convention deviations and indentation errors.
- **AIPredictor (Probabilistic):** Uses a **CodeBERT** vectorizer to perform pattern matching against historical bug data, allowing it to predict **Logic Bugs** and provide **Refactoring Tips** that standard rules might miss.

4. Finding Generation & Aggregation

As the engines finish their work, they each "output" discrete **Finding** objects.

- **Unified Collection:** These findings are funneled into a temporary list within the orchestrator.
- **Data Structure:** Each finding at this stage includes the **line_number**, **severity**, and a **message** describing the issue.

5. Handoff to ReportBuilder

The flowchart concludes with the list of findings being handed off to the **ReportBuilder**. This final component is responsible for calculating the **Overall Health Score** and preparing the data for both the **Supabase** database and the **Frontend** dashboard.

